SHERLOCK

Security Review For M0

Collaborative Audit Prepared For:	**M0**
Lead Security Expert(s):	**defsec** **EgisSecurity**
Date Audited:	**December 9 - December 19, 2025**

Introduction

M0's Portal V2 is the second major iteration of our native crosschain interoperability solution. It provides a unified interface for sending ecosystem tokens and data between EVM and SVM blockchains. It provides an abstraction for different messaging providers to allow extensibility and flexibility to meet our customers' requirements. It is also a foundational building block for our liquidity solutions.

Scope

Repository: m0-foundation/m-portal-v2

Audited Commit: b9799699cab6986e09beee4833423510d3128c1d

Final Commit: fc69ff4cf1b68017f45f3d53242f57ed3e5e4973

Files:

- evm/src/bridgeAdapters/BridgeAdapter.sol
- evm/src/bridgeAdapters/hyperlane/HyperlaneBridgeAdapter.sol
- evm/src/bridgeAdapters/hyperlane/interfaces/IHyperlaneBridgeAdapter.sol
- evm/src/bridgeAdapters/wormhole/interfaces/IWormholeBridgeAdapter.sol
- evm/src/bridgeAdapters/wormhole/WormholeBridgeAdapter.sol
- evm/src/HubPortal.sol
- evm/src/interfaces/IBridgeAdapter.sol
- evm/src/interfaces/IHubPortal.sol
- evm/src/interfaces/IMTokenLike.sol
- evm/src/interfaces/IOrderBookLike.sol
- evm/src/interfaces/IPortal.sol
- evm/src/interfaces/IRegistrarLike.sol
- evm/src/interfaces/ISpokeMTokenLike.sol
- evm/src/interfaces/ISpokePortal.sol
- evm/src/interfaces/ISwapFacilityLike.sol
- evm/src/libraries/BytesParser.sol
- evm/src/libraries/PayloadEncoder.sol
- evm/src/libraries/TypeConverter.sol
- evm/src/Portal.sol
- evm/src/SpokePortal.sol

- evm/src/utils/ReentrancyLock.sol

Repository: m0-foundation/solana-portal

Audited Commit: 687ba581e11593abc975f48d296a39b28c5a4661

Final Commit: eaa029e11c493d909149cd8ea8e2dd4c473eae4d

Files:

- packages/common/Cargo.toml
- packages/common/idls/earn.json
- packages/common/idls/ext_swap.json
- packages/common/idls/hyperlane_adapter.json
- packages/common/idls/order_book.json
- packages/common/idls/portal.json
- packages/common/idls/wormhole_adapter.json
- packages/common/idls/wormhole_post_message_shim.json
- packages/common/idls/wormhole_verify_vaa_shim.json
- packages/common-macros/Cargo.toml
- packages/common-macros/src/lib.rs
- packages/common/src/accounts.rs
- packages/common/src/adapter_accounts.rs
- packages/common/src/conversions.rs
- packages/common/src/errors.rs
- packages/common/src/interfaces.rs
- packages/common/src/lib.rs
- packages/common/src/payloads.rs
- packages/common/src/receive_metas.rs
- programs/hyperlane-adapter/Cargo.toml
- programs/hyperlane-adapter/src/consts.rs
- programs/hyperlane-adapter/src/instructions/get_ism.rs
- programs/hyperlane-adapter/src/instructions/initialize.rs
- programs/hyperlane-adapter/src/instructions/mod.rs
- programs/hyperlane-adapter/src/instructions/pause.rs

- programs/hyperlane-adapter/src/instructions/receive_message.rs
- programs/hyperlane-adapter/src/instructions/send_message.rs
- programs/hyperlane-adapter/src/instructions/set_igp_gas_amount.rs
- programs/hyperlane-adapter/src/instructions/set_igp.rs
- programs/hyperlane-adapter/src/instructions/set_ism.rs
- programs/hyperlane-adapter/src/instructions/set_peer.rs
- programs/hyperlane-adapter/src/instructions/sync_extensions.rs
- programs/hyperlane-adapter/src/instructions/transfer_admin.rs
- programs/hyperlane-adapter/src/lib.rs
- programs/hyperlane-adapter/src/state.rs
- programs/portal/Cargo.toml
- programs/portal/src/instructions/initialize.rs
- programs/portal/src/instructions/mod.rs
- programs/portal/src/instructions/pause.rs
- programs/portal/src/instructions/receive_message.rs
- programs/portal/src/instructions/send_fill_report.rs
- programs/portal/src/instructions/send_index.rs
- programs/portal/src/instructions/send_merkle_root.rs
- programs/portal/src/instructions/send_token.rs
- programs/portal/src/instructions/transfer_admin.rs
- programs/portal/src/lib.rs
- programs/portal/src/state.rs
- programs/portal/Xargo.toml
- programs/wormhole-adapter/Cargo.toml
- programs/wormhole-adapter/src/consts.rs
- programs/wormhole-adapter/src/instructions/initialize.rs
- programs/wormhole-adapter/src/instructions/mod.rs
- programs/wormhole-adapter/src/instructions/pause.rs
- programs/wormhole-adapter/src/instructions/receive_message.rs
- programs/wormhole-adapter/src/instructions/resolve_execute.rs
- programs/wormhole-adapter/src/instructions/send_message.rs

- programs/wormhole-adapter/src/instructions/set_lut.rs
- programs/wormhole-adapter/src/instructions/set_peer.rs
- programs/wormhole-adapter/src/instructions/transfer_admin.rs
- programs/wormhole-adapter/src/lib.rs
- programs/wormhole-adapter/src/state.rs
- programs/wormhole-adapter/Xargo.toml

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	4	13

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Index size mismatch between EVM (128-bit) and Solana (64-bit) causes payload encoding incompatibility [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/86

Summary

The M token index is stored as 128 bits (uint128) in EVM but only 64 bits (u64) in Solana. The payload encoding differs between chains => EVM encodes index as 16 bytes while Solana expects 8 bytes. This creates a cross-chain payload incompatibility that will cause message parsing failures or data corruption when transferring between EVM and Solana.

Vulnerability Detail

EVM TokenTransfer Payload Layout (Portal.sol):

```
[0]       PayloadType (1 byte)
[1-16]    amount (uint128 - 16 bytes)
[17-48]   destinationToken (bytes32 - 32 bytes)
[49-80]   sender (bytes32 - 32 bytes)
[81-112]  recipient (bytes32 - 32 bytes)
[113-128] index (uint128 - 16 bytes)
[129-160] messageId (bytes32 - 32 bytes)
Total: 161 bytes
```

Solana TokenTransfer Payload Layout (payloads.rs):

```
[0]       PayloadType (1 byte)
[1-16]    amount (u128 - 16 bytes)
[17-48]   destination_token (32 bytes)
[49-80]   sender (32 bytes)
[81-112]  recipient (32 bytes)
[113-120] index (u64 - 8 bytes)      ← Mismatch (8 bytes vs 16 bytes)
[121-152] message_id (32 bytes)
Total: 153 bytes
```

In Solana's payload decoding:

```
let (index_bytes, data) = data.split_at(8);  // Only reads 8 bytes for index
```

While the current index value fits in u64, if the index ever exceeds `u64::MAX`, the Solana side would truncate the value, causing incorrect yield calculations.

Impact

Payload length and field offset mismatch causes parsing errors.

Code Snippet

EVM (PayloadEncoder.sol) - Uses uint128 (16 bytes):

```
function encodeTokenTransfer(
    uint256 amount,
    bytes32 destinationToken,
    bytes32 sender,
    bytes32 recipient,
    uint128 index,        // 16 bytes
    bytes32 messageId
) internal pure returns (bytes memory) {
    return abi.encodePacked(
        PayloadType.TokenTransfer,
        uint128(amount),
        destinationToken,
        sender,
        recipient,
        index,            // Encoded as 16 bytes
        messageId
    );
}
```

Solana (payloads.rs) - Uses u64 (8 bytes):

```
pub struct TokenTransferPayload {
    pub amount: u128,
    pub destination_token: [u8; 32],
    pub sender: [u8; 32],
    pub recipient: [u8; 32],
    pub index: u64,       // Only 8 bytes
    pub message_id: [u8; 32],
}

impl TokenTransferPayload {
    pub fn decode(data: &[u8]) -> Result<Self> {
        // ... earlier fields ...
        let (index_bytes, data) = data.split_at(8);  // Only reads 8 bytes!
        let index = u64::from_be_bytes(index_bytes.try_into().unwrap());
        // ...
    }
}
```

Tool Used

Manual Review

Recommendation

Align the index size across both platforms. Since EVM uses `uint128`, update Solana to use `u128`.

Discussion

SC4RECOIN

This also impacts `IndexPayload`

Issue M-1: Hyperlane adapter relies on default ism rather than custom security module [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/92

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The Hyperlane bridge adapters on both EVM and Solana rely on Hyperlane's default Interchain Security Module (ISM) rather than specifying a custom one.

Vulnerability Detail

On EVM, the `HyperlaneBridgeAdapter` explicitly returns zero address for the `interchainSecurityModule` function, causing the Mailbox to use the default ISM for all message verification.

On Solana, the `HyperlaneGlobal` state initializes with ism set to `None`. While a set_ism function exists allowing admin to configure a custom ISM, the default deployment uses None which falls back to Hyperlane's default ISM.

Reference : https://github.com/hyperlane-xyz/hyperlane-monorepo/blob/main/solidity/contracts/isms/NoopIsm.sol

Impact

The protocol inherits whatever security guarantees the default ISM provides, which could change without M0's control.

Tool Used

Manual Review

Recommendation

Configure a custom ISM tailored to the protocol's security requirements. On EVM, implement `interchainSecurityModule` to return a configured ISM address rather than zero. On Solana, set a specific ISM during initialization or immediately after deployment via `set_ism`.

Issue M-2: `Portal.sol` - it is possible to use unsupported bridge adaptor, if it was the default portal [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/94

Summary

If an operator removes the default bridge adaptor in `Portal.sol`, it is still possible to be used with the functions that fetch it using `defaultBridgeAdapter(destinationChainId)`:

```
    address bridgeAdapter = defaultBridgeAdapter(destinationChainId);
    _revertIfZeroBridgeAdapter(destinationChainId, bridgeAdapter);

    return
        _sendToken(
            amount, sourceToken, destinationChainId, destinationToken, recipient,
            ↪  refundAddress, bridgeAdapter, bridgeAdapterArgs
        );
}
```

This may lead to using compromised bridge adapter after operator function to delist it.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/fcfbb779e229eb3e80f58305c270af2e65dbd628/m-portal-v2/evm/src/Portal.sol#L207-L217

Tool Used

Manual Review

Recommendation

In `setSupportedBridgeAdapter` if `supported == false`, check if that's the default bridge, if so... reset `remoteChainConfig.defaultBridgeAdapter`, or revert

Issue M-3: Stack `_sendRegistrarListStatus` messages with account removal to execute them later on dest chain [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/96

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The HubPortal contract allows any user to broadcast the current registrar list status of an account to a destination chain via the `sendRegistrarListStatus` function. This function retrieves the current status from the registrar (via listContains), encodes it into a payload (containing listName, account, add status), and transmits it via the selected bridge adapter (e.g., Wormhole). Crucially, the `RegistrarList` payload generated by `PayloadEncoder.encodeRegistrarList` does not include a timestamp, block number, or deadline. Furthermore, the `WormholeBridgeAdapter` and `SpokePortal` on the destination chain do not enforce sequential execution of messages based on their creation time; they only enforce replay protection for identical message hashes. This lack of temporal enforcement allows an attacker to "stack" or hoard valid cross-chain messages that set an account's status to false (removed) and execute them at a later time. By providing a low execution fee (or 0) to the executor during the `sendRegistrarListStatus` call, an attacker can ensure the message is published to the Wormhole network but not immediately relayed by the default executor. The attacker can then hold this valid VAA (Verified Action Approval) indefinitely.

Impact

An attacker can grief any user who is expected to receive permissions (e.g., "Earner" status) on a destination chain

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/fcfbb779e229eb3e80f58305c270af2e65dbd628/m-portal-v2/evm/src/HubPortal.sol#L105-L129

Tool Used

Manual Review

Recommendation

Validate `CoreBridgeVM.timestamp` on dest for a valid timestamp

Discussion

0xIryna

We accept this limitation since Registrar Key and Registrar List Updates messages aren't frequent and anybody can propagate the correct Key or List Status

Issue M-4: It is possible to send higher index to dest chain from `HubPortal` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/101

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`HubPortal` implements `enableEarning` and `disableEarning` functions to start/stop earnings from mToken and update the `HubPortalStorageStruct` vars

When a `MtokenIndex` is send and the earning is disabled, we fetch the index from `_getHubPortalStorageLocation`, which is updated once `disableEarning` is invoked. However, it is possible that the Hub contract is delisted from earners and disabled trough directly calling `mToken.stopEarning(hubContract address)`, without saving the correct index in `HubPortal` storage. If later on `disableEarning` is called on the Hub, it will safe larger index on dest chain, when it shouldn't.

Impact

Broken solvency because if all bridged tokens on dest chain are earning, we won't have enough balance on source to release when bridged back

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/fcfbb779e229eb3e80f58305c270af2e65dbd628/m-portal-v2/evm/src/HubPortal.sol#L163

Tool Used

Manual Review

Recommendation

I am not sure what is the best mitigation here. One solution is to have an off-chain bot listening for events from `MToken` and call `disableEarning` on `HubPortal`, if someone disable `HubPortal` contract from earning

Discussion

0xIryna

The likelihood of `HubPortal` being removed from the earner list is extremely low. If it happens, M0 team will call `disableEarning` on `HubPortal`

Issue L-1: EVM portal `receiveMessage` function missing `whenNotPaused` modifier [RESOLVED]

Source:
https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/84

Summary

The EVM Portal's `receiveMessage` function lacks the `whenNotPaused` modifier, allowing cross-chain messages to be processed even when the portal is paused. This creates an asymmetric pause mechanism where outbound operations are blocked but inbound operations continue, undermining the emergency pause functionality.

Vulnerability Detail

In `Portal.sol`, the `receiveMessage` function does not include the `whenNotPaused` modifier:

```
/// @inheritdoc IPortal
function receiveMessage(uint32 sourceChainId, bytes calldata payload) external {
    _revertIfUnsupportedBridgeAdapter(sourceChainId, msg.sender);

    PayloadType payloadType = payload.getPayloadType();

    if (payloadType == PayloadType.TokenTransfer) {
        _receiveToken(sourceChainId, payload);
        return;
    }

    if (payloadType == PayloadType.FillReport) {
        _receiveFillReport(sourceChainId, payload);
        return;
    }

    // Index, Registrar Key, or Registrar List Update on Spoke chains
    _receiveCustomPayload(payloadType, payload);
}
```

In contrast, all send functions include both `whenNotPaused` and `whenNotLocked` modifiers:

```
function sendToken(
    uint256 amount,
    address sourceToken,
    uint32 destinationChainId,
    bytes32 destinationToken,
    bytes32 recipient,
    bytes32 refundAddress,
```

```
    bytes calldata bridgeAdapterArgs
) external payable whenNotPaused whenNotLocked returns (bytes32 messageId) {
    // ...
}

function sendFillReport(
    uint32 destinationChainId,
    IOrderBookLike.FillReport calldata report,
    bytes32 refundAddress,
    bytes calldata bridgeAdapterArgs
) external payable whenNotPaused whenNotLocked returns (bytes32 messageId) {
    // ...
}
```

Impact

Pause mechanism cannot fully halt operations during security incidents.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/main/m-portal-v2/evm/src/Portal.sol#L165

Tool Used

Manual Review

Recommendation

Add the `whenNotPaused` modifier to the `receiveMessage` function.

Discussion

0xIryna

Thank you for reporting the issue!

If we pause the Portals for an upgrade, we still want in-flight messages to be processed on the destination. In the case of an exploit, an operator could update `supportedBridgeAdapter` mapping in `Portal` or `remotePeer` mapping in a `BridgeAdapter` to force `receiveMessage` to revert, though this becomes more cumbersome when multiple adapters are used.

Do you have any recommendations on the best way to handle this scenario?

defsec

Thank you for the clarification!

Instead of modifying `supportedBridgeAdapter` mappings (which loses config), what about adding a separate `freezeAdapter` function in `Portal`? You could have a `frozenAdapters` mapping and check it in `_revertIfUnsupportedBridgeAdapter`. This way you can emergency-block receives without losing adapter configuration and it's easily reversible. What do you think?

0xIryna

We added separate flags and modifiers for sending and receiving. This approach provides more granular control over pausing operations. Here is the PR

Issue L-2: `setBridgeChainId` silently overwrites reverse mapping without collision check [RESOLVED]

Source:
https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/85

Summary

The `setBridgeChainId` function in `BridgeAdapter.sol` creates a bidirectional mapping between internal chain IDs and bridge-specific chain IDs without checking for collisions in the reverse mapping. If an operator maps two different internal chain IDs to the same bridge chain ID, the reverse mapping (`bridgeToInternalChainId`) is silently overwritten, breaking the first chain's receive path.

Vulnerability Detail

In `BridgeAdapter.sol`:

```
function setBridgeChainId(uint32 chainId, uint256 bridgeChainId) external
↪  onlyRole(OPERATOR_ROLE) {
    _revertIfZeroChain(chainId);
    _revertIfZeroBridgeChain(bridgeChainId);

    BridgeAdapterStorageStruct storage $ = _getBridgeAdapterStorageLocation();

    if ($.internalToBridgeChainId[chainId] == bridgeChainId) return;

    $.internalToBridgeChainId[chainId] = bridgeChainId;
    $.bridgeToInternalChainId[bridgeChainId] = chainId;  // <-- No collision check!

    emit BridgeChainIdSet(chainId, bridgeChainId);
}
```

The storage structure maintains two mappings:

```
struct BridgeAdapterStorageStruct {
    /// @notice Maps M0 internal chain IDs to bridge-specific chain IDs.
    mapping(uint32 internalChainId => uint256 bridgeChainId)
    ↪  internalToBridgeChainId;
    /// @notice Maps M0 internal chain IDs to remote Bridge Adapter addresses.
    mapping(uint32 internalChainId => bytes32 peer) remotePeer;
    /// @notice Maps bridge-specific chain IDs to M0 internal chain IDs.
    mapping(uint256 bridgeChainId => uint32 internalChainId)
    ↪  bridgeToInternalChainId;
}
```

Scenario:

1. Operator calls `setBridgeChainId(1, 100)` - Maps internal chain 1 to bridge chain 100
    - `internalToBridgeChainId[1] = 100`
    - `bridgeToInternalChainId[100] = 1`
2. Operator accidentally calls `setBridgeChainId(2, 100)` - Maps internal chain 2 to same bridge chain 100
    - `internalToBridgeChainId[2] = 100`
    - `bridgeToInternalChainId[100] = 2` (overwrites previous value!)
3. Now the mappings are inconsistent:
    - `internalToBridgeChainId[1] = 100` (send path for chain 1 works)
    - `internalToBridgeChainId[2] = 100` (send path for chain 2 works)
    - `bridgeToInternalChainId[100] = 2` (receive path only resolves to chain 2!)

Impact

No error or warning when collision occurs.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/pull/1/files/c059ac15774f99a3dc289e37971144a7e2e3f3ea

Tool Used

Manual Review

Recommendation

Add a collision check for the reverse mapping before overwriting.

Issue L-3: Off-by-one error in hyperlane remaining accounts check causes index out of bounds panic [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/87

Summary

The Solana Portal's `send_message` function contains an off-by-one error when validating the length of `remaining_accounts` for the Hyperlane adapter path. The code checks if `remaining_accounts.len() < 11` but then accesses `remaining_accounts[11]`, which requires at least 12 elements. This causes an index out of bounds panic when exactly 11 accounts are provided.

Vulnerability Detail

```
} else if bridge_adapter.key() == common::hyperlane_adapter::ID {
    if remaining_accounts.len() < 11 {
        return err!(BridgeError::InvalidRemainingAccounts);
    }

    // Delegate account validation to hyperlane adapter
    let hyperlane_global = remaining_accounts[0].clone();
    let mailbox_outbox = remaining_accounts[1].clone();
    let dispatch_authority = remaining_accounts[2].clone();
    let hyperlane_user_global = remaining_accounts[3].clone();
    let unique_message = remaining_accounts[4].clone();
    let dispatched_message = remaining_accounts[5].clone();
    let igp_program_id = remaining_accounts[6].clone();
    let igp_program_data = remaining_accounts[7].clone();
    let igp_gas_payment = remaining_accounts[8].clone();
    let igp_account = remaining_accounts[9].clone();
    let mailbox_program = remaining_accounts[10].clone();
    let spl_noop_program = remaining_accounts[11].clone();  // @audit Index 11
    ↪  requires len >= 12!

    // Account is optional
    let igp_overhead_account = remaining_accounts.get(12).cloned();
```

The Problem:

1. The check `remaining_accounts.len() < 11` passes when `len == 11`
2. When `len == 11`, valid indices are `0..10` (inclusive)

3. The code accesses `remaining_accounts[11]` which is out of bounds
4. This causes a panic (index out of bounds)

Account Index Analysis:

Index	Account	Required?
0	hyperlane_global	Yes
1	mailbox_outbox	Yes
2	dispatch_authority	Yes
3	hyperlane_user_global	Yes
4	unique_message	Yes
5	dispatched_message	Yes
6	igp_program_id	Yes
7	igp_program_data	Yes
8	igp_gas_payment	Yes
9	igp_account	Yes
10	mailbox_program	Yes
11	spl_noop_program	Yes
12	igp_overhead_account	Optional

The code needs 12 mandatory accounts (indices 0-11), so the check should be `len < 12`.

Impact

Transactions with 11 accounts to cause panics.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/main/solana-portal/programs/portal/src/instructions/mod.rs#L91

Tool Used

Manual Review

Recommendation

Use the same pattern as Wormhole.

Discussion

SC4RECOIN

would argue this is a low there is no impact here a panic means the instruction was called wrong and the transaction fails a panic instead of an error here is bad ux but nothing more

defsec

Got it, thank you for feedback, marking as a low.

Issue L-4: Missing `earn_program` validation in token transfer payload accounts [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/88

Summary

The `TokenTransferPayload` validation function does not verify that the `earn_program` account matches the expected earn program ID. While IndexPayload and `EarnerMerkleRootPayload` both validate the earn_program key against the known `earn::ID`, `TokenTransferPayload` skips this check despite including `earn_program` in its accounts struct.

Vulnerability Detail

The `IndexPayload` validation correctly checks that the `earn_program` matches the expected program ID before returning the accounts. The same validation exists for `EarnerMerkleRootPayload`. However, `TokenTransferPayload` includes `earn_program` as one of its required accounts but the `parse_and_validate_accounts` function only validates the `recipient_token_account` derivation and the `swap_program` key.

This inconsistency means that when processing a `TokenTransfer` payload, a malicious relayer could provide an arbitrary program as the `earn_program` account. The `TokenTransferPayloadAccounts` struct shares the first four fields with `IndexPayloadAccounts`, including `m_global`, `m_mint`, `earn_program`, and `m_token_program`. These accounts are used when the token transfer also triggers an index update through the `handle_index_payload` function.

Impact

An attacker controlling the relayer could substitute a malicious program for `earn_program` when relaying TokenTransfer messages.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/main/solana-portal/packages/common/src/accounts.rs#L76

Tool Used

Manual Review

Recommendation

Add the `earn_program` validation to `TokenTransferPayload's` `parse_and_validate_accounts` function to match the pattern used by `IndexPayload` and `EarnerMerkleRootPayload`.

Discussion

SC4RECOIN

Good catch but I would argue this is a low as the account still gets validated in this call https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/main/solana-portal/programs/portal/src/instructions/receive_message.rs allowbreak #L128

this account is only used when propagating indexes and is validated when doing so

defsec

Downgraded that one as a low , thank you for feedback!

Issue L-5: Unbounded peer vector growth in wormhole adapter state [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/89

Summary

The Wormhole adapter's WormholeGlobal state stores peers in a Vec that can only grow via `set_peer`. There is no function to remove peers, leading to unbounded state growth, increased rent costs, and slower peer validation lookups over time.

Vulnerability Detail

The `WormholeGlobal` account stores peers in an unbounded vector. The `set_peer` instruction allows adding or updating peers but no `remove_peer` instruction exists. Each Peer struct consumes 36 bytes (32-byte address + 4-byte chain_id).

The validate and `get_peer` functions perform linear searches through all peers on every message, adding compute overhead that scales with peer count. The same pattern exists in `HyperlaneGlobal`.

Impact

Limited impact since the peer set represents supported chains which is inherently bounded. Admin-only operation prevents malicious growth. Main concerns are operational inefficiency if deprecated chains accumulate and slightly higher rent costs.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/fcfbb779e229eb3e80f58305c270af2e65dbd628/solana-portal/programs/wormhole-adapter/src/lib.rs#L57

Tool Used

Manual Review

Recommendation

Add a `remove_peer` instruction allowing admin to delete peers for deprecated chains.

Issue L-6: `destination_token` in payload not validated against `extension_mint` in remaining accounts [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/90

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When receiving a token transfer on Solana, the `destination_token` specified in the cross-chain payload is not validated against the `extension_mint` account passed in remaining_accounts. A relayer could substitute a different whitelisted extension mint than what the user originally specified.

Vulnerability Detail

The `TokenTransferPayload` contains a `destination_token` field representing the user's desired extension token. During `receive_message` processing, the `extension_mint` is extracted from `remaining_accounts` but never compared against the payload's `destination_token`.

The `parse_and_validate_accounts` function validates that `recipient_token_account` matches the expected ATA and that `swap_program` matches `ext_swap::ID`, but does not verify that `extension_mint` equals `destination_token` from the payload.

A relayer constructing the `remaining_accounts` could provide a different whitelisted extension mint than what the user specified in the original cross-chain message.

Impact

Relayers can only substitute with extensions whitelisted in the swap program, and all whitelisted extension tokens can be swapped 1:1 with each other. A malicious substitution would not benefit the relayer and would only potentially cause user confusion about which extension token they received.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/fcfbb779e229eb3e80f58305c270af2e65dbd628/solana-portal/packages/common/src/accounts.rs#L20

Tool Used

Manual Review

Recommendation

Consider adding validation that `accounts.extension_mint.key` equals `Pubkey::from(self.destination_token)` in the `parse_and_validate_accounts` function for `TokenTransferPayload`. Alternatively, document this as accepted behavior given the 1:1 swap capability between whitelisted extensions.

Discussion

SC4RECOIN

not fixing as stated here
https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/pull/33

Issue L-7: Permissionless initialization allows any signer to become admin [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/91

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

All three Solana programs (Portal, Hyperlane Adapter, and Wormhole Adapter) allow any signer to become the admin during initialization. While the PDA ensures the global account can only be initialized once, whoever calls the initialize instruction first becomes the permanent admin with full control over the protocol.

Vulnerability Detail

The Portal program's initialize instruction accepts any signer as the admin parameter. The instruction creates a PDA-based global account and sets the caller as the admin. Since the PDA can only be initialized once, the first caller permanently becomes the admin.

Impact

Permissionless initialization allows any signer to become admin.

Tool Used

Manual Review

Recommendation

The recommended fix is to add an authorized deployer check to all initialize functions. This can be implemented by hardcoding a known deployer public key in the program and requiring that the admin signer matches this address.

Issue L-8: Wormhole adapter `SetLookupTable` instruction missing admin access control [RESOLVED]

Source:
https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/93

Summary

The `SetLookupTable` instruction in the Wormhole adapter lacks an `admin` signer requirement and `has_one` constraint.

Vulnerability Detail

The `SetLookupTable` accounts struct only requires a payer signer but does not verify the caller is the admin. Compare with SetPeer which correctly enforces admin access control with both an admin signer and has_one constraint.

Any user can call `SetLookupTable` with any `recent_slot` value, creating a new LUT and overwriting the `receive_lut` stored in `wormhole_global`.

Impact

The previous LUT remains valid even if a new one is set. The instruction only affects the `receive_message` instruction's `resolve_execute` path, and updating the LUT does not invalidate existing LUTs that may be referenced.

Tool Used

Manual Review

Recommendation

While adding admin access control would be more consistent with other admin functions, the lack of it does not pose a security risk as old LUTs remain valid.

Issue L-9: `assert_eq!` causes panic [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/95

Summary

The Hyperlane adapter's `send_message` handler uses the `assert_eq!` macro to validate the returning program ID after a CPI call. In Solana programs, `assert_eq!` causes a panic which aborts the transaction ungracefully instead of returning a proper Anchor error.

Vulnerability Detail

After making a CPI call to the Hyperlane mailbox program, the code uses `assert_eq!` to verify that the return data came from the expected program. The `assert_eq!` macro in Rust triggers a panic when the assertion fails, which in Solana programs results in an ungraceful transaction abort with a generic "Program failed" error message rather than a descriptive Anchor error.

Impact

When the assertion fails, users and developers see a generic panic message instead of a descriptive error code.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/fcfbb779e229eb3e80f58305c270af2e65dbd628/solana-portal/programs/hyperlane-adapter/src/instructions/send_message.rs#L202

Tool Used

Manual Review

Recommendation

Replace `assert_eq!` with Anchor's `require!` macro to return a proper program error.

Issue L-10: IGP overhead account constraint to handle `None` [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/97

Summary

The Hyperlane adapter's `send_message` instruction uses `.unwrap()` on an `Option<Pubkey>` field within an account constraint, which can panic if the optional account is provided but the global state field is None.

Vulnerability Detail

The `igp_overhead_account` is an `optional` account with a constraint that validates its address against `hyperlane_global.igp_overhead_account`:

```
#[account(
    owner = igp_program_id.key(),
    address = hyperlane_global.igp_overhead_account.unwrap() @
    ↪  BridgeError::InvalidIgpAccount,
)]
pub igp_overhead_account: Option<AccountInfo<'info>>,
```

When a user provides this optional account, Anchor evaluates the constraints. If `hyperlane_global.igp_overhead_account` is `None`, the `.unwrap()` call panics before the custom error `BridgeError::InvalidIgpAccount` can be returned.

Impact

Could cause a panic.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/main/solana-portal/programs/hyperlane-adapter/src/instructions/send_message.rs#L132

Tool Used

Manual Review

Recommendation

Use unwrap_or_default() which will fail the address check gracefully.

Discussion

SC4RECOIN

will update to

```
constraint = hyperlane_global.igp_overhead_account ==
↪  Some(igp_overhead_account.key()) @ BridgeError::InvalidIgpAccount,
```

Issue L-11: `Payload::decode` returns self instead of `Result` [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/98

Summary

The `Payload::decode` function returns `Self` instead of `Result<Self>`, making graceful error handling impossible at call sites. This design forces the function to panic on any malformed input rather than propagating errors.

Vulnerability Detail

The `decode` function has multiple panic points that cannot be handled by callers:

```
pub fn decode(data: &Vec<u8>) -> Self {  // Returns Self, not Result<Self>!
    let (payload_type, data) = data.split_at(1);  // Panics if empty

    match payload_type[0] {
        Self::TOKEN_TRANSFER_DISCRIMINANT => {
            let (amount_bytes, data) = data.split_at(16);  // Panics if < 16 bytes
            let (destination_token_bytes, data) = data.split_at(32);  // Panics if
            ↪  < 32 bytes
            // ... more split_at calls ...

            Payload::TokenTransfer(TokenTransferPayload {
                amount: u128::from_be_bytes(amount_bytes.try_into().unwrap()),  //
                ↪  Unwrap panics
                destination_token: destination_token_bytes.try_into().unwrap(),  //
                ↪  Unwrap panics
                // ...
            })
        }
        // ... other discriminants ...
        _ => panic!("Invalid payload type"),  // Explicit panic!
    }
}
```

Impact

Panics produce generic errors instead of descriptive error codes.

Code Snippet

Location: `packages/common/src/payloads.rs:54-118`

```
pub fn decode(data: &Vec<u8>) -> Self {
    let (payload_type, data) = data.split_at(1);

    match payload_type[0] {
        Self::TOKEN_TRANSFER_DISCRIMINANT => {
            // Multiple split_at and unwrap calls that can panic
            let (amount_bytes, data) = data.split_at(16);
            // ...
            Payload::TokenTransfer(TokenTransferPayload {
                amount: u128::from_be_bytes(amount_bytes.try_into().unwrap()),
                // ...
            })
        }
        // ...
        _ => panic!("Invalid payload type"),  // Line 116
    }
}
```

Tool Used

Manual Review

Recommendation

Change the function signature to return `Result<Self>` and use proper error handling.

Issue L-12: `HyperlaneGlobal::size()` allocates 8 Bytes for non-existent nonce field [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/99

Summary

The `HyperlaneGlobal::size()` function includes 8 bytes for a nonce field that does not exist in the HyperlaneGlobal struct, causing over-allocation of account space and wasted rent.

Vulnerability Detail

The HyperlaneGlobal struct definition does not contain a nonce field, but the corresponding `size()` function includes 8 bytes allocated for nonce in its calculation. This appears to be a copy-paste error from HyperlaneUserGlobal, which does have a nonce field.

The struct contains 11 fields:

- bump (u8): 1 byte
- admin (Pubkey): 32 bytes
- paused (bool): 1 byte
- igp_program_id (Pubkey): 32 bytes
- igp_gas_amount (u64): 8 bytes
- igp_account (Pubkey): 32 bytes
- igp_overhead_account (Option): 33 bytes
- ism (Option): 33 bytes
- pending_admin (Option): 33 bytes
- peers (Vec): 4 + n×36 bytes
- padding ([u8; 128]): 128 bytes

Total with discriminator: 345 + n×36 bytes

However, `size()` returns: 353 + n×36 bytes (8 bytes more than actual struct size)

Impact

Every `HyperlaneGlobal` account pays rent for 8 extra bytes that are never used.

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/main/solana-portal/programs/hyperlane-adapter/src/state.rs#L37

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/main/solana-portal/programs/hyperlane-adapter/src/state.rs#L62

Tool Used

Manual Review

Recommendation

Remove the phantom nonce allocation from the `size()` function.

Issue L-13: Potential locked tokens when edge case mint revert appears in `SpokeMTokenL` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/issues/100

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

Mtoken::_mint reverts under the following condition:

```
unchecked {
    // As an edge case precaution, prevent a mint that, if all tokens (earning and
     ↪  non-earning) were converted
    // to a principal earning amount, would overflow the `uint112
     ↪  principalOfTotalEarningSupply`.
    if (
        uint256(totalNonEarningSupply) + safeAmount_ > type(uint240).max ||
        // NOTE: Round the principal up for worst case.
        uint256(principalOfTotalEarningSupply) +
            _getPrincipalAmountRoundedUp(totalNonEarningSupply + safeAmount_) >=
        type(uint112).max
    ) {
        revert OverflowsPrincipalOfTotalSupply();
    }
}
```

Which means that user may end up in a state, where he has locked his tokens on source and the minting on dest chain is blocked.

Impact

Permanent DoS of users funds

Code Snippet

https://github.com/sherlock-audit/2025-12-m-0-multichain-v2-dec-8th/blob/fcfbb779e229eb3e80f58305c270af2e65dbd628/m-portal-v2/evm/src/SpokePortal.sol#L93-L95

Tool Used

Manual Review

Recommendation

Consider wrapping `_mintOrUnlock` function on the spoke chain in a try/catch block and initiating a bridge message to unlock the tokens on source, if revert appears.

Discussion

0xIryna

It's unrealistic for $M supply to surpass `type(uint112).max` considering that $M is backed 1:1 by US T-bills

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.